

08

CLOUD COMPUTING COMO PILAR DA IOT MODERNA

PDF exclusivo para Murilo Salla - rm568041
murilosalla@protonmail.com

LISTA DE FIGURAS

Figura 1 - Arquitetura distribuída em sistemas ciberfísicos com camadas de borda, névoa e nuvem 10
Figura 2 - Arquitetura de computação em nuvem com camadas de infraestrutura, plataforma e aplicação em IoT 17
Figura 3 - Arquitetura da experiência de computação em nuvem com ESP32 e plataformas integradas 18
Figura 4 - Diagrama de conexão entre ESP32 DevKit V4 e MPU6050 via I²C 20
Figura 5 - Tela inicial do HiveMQ Cloud com cluster gratuito configurado 24
Figura 6 - Definição de credenciais MQTT no painel de gerenciamento do HiveMQ Cloud 25
Figura 7 - Configuração do nó MQTT de entrada no Node-RED para conexão com HiveMQ Cloud 26
Figura 8 - Configuração dos nós MQTT no Node-RED: tópico e servidor 27
Figura 9 - Configuração do nó HTTP para envio dos dados à InfluxDB 28
Figura 10 - Bucket “Cloud-Computing” criado no InfluxDB Cloud 29
Figura 11 - Consulta dos dados no Data Explorer do InfluxDB Cloud 30
Figura 12 - Configuração da fonte de dados InfluxDB no Grafana Cloud 31
Figura 13 - Visualização dos dados de aceleração em tempo real no Grafana Cloud 32

FIAP

LISTA DE QUADROS

Quadro 1 - Vantagens e desvantagens da computação em nuvem.....12

FIAP

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Inclusão de bibliotecas e definição de parâmetros 21
Código-fonte 2 - Conexão à nuvem e inicialização de sensores.....22
Código-fonte 3 - Leitura dos dados do MPU6050 23
Código-fonte 4 - Formatação e envio dos dados para a nuvem.....23

FIAP

SUMÁRIO

1 CLOUD COMPUTING COMO PILAR DA IOT MODERNA 6
1.1 Fundamentos da computação distribuída em IoT 6
1.2 Conceito de Cloud, Fog e Edge Computing 7
1.3 Arquiteturas distribuídas em sistemas ciberfísicos 9
1.4 Vantagens e desvantagens da computação em nuvem 11
1.4.1 Casos de uso típicos em IoT 13

2 CLOUD COMPUTING COM ESP32 E GRAFANA CLOUD 15
2.1 Fundamentos da computação em nuvem aplicada à IoT 15
2.2 Experiência prática com ESP32 e visualização em nuvem 17
2.2.1 Hardware – ESP32 e MPU6050 18
2.2.2 Transmissão MQTT – Broker HiveMQ Cloud 23
2.2.3 Processamento e encaminhamento – Node-RED 25
2.2.4 Armazenamento de dados – InfluxDB Cloud 28
2.2.5 Visualização e dashboards – Grafana Cloud 30

3 COMPARAÇÃO ENTRE NUVEM, NÉVOA E BORDA 33
3.1 Potencial da computação em nuvem 33
3.2 Quais são as limitações e desafios da computação em nuvem? 35
3.3 Estratégias para maximizar o uso da nuvem 36

REFERÊNCIAS 38

FATEC

1 CLOUD COMPUTING COMO PILAR DA IOT MODERNA

1.1 Fundamentos da computação distribuída em IoT

O avanço da Internet das Coisas (IoT) tem promovido mudanças significativas nas formas tradicionais de processamento e gerenciamento de dados. Diferentemente dos sistemas centralizados, onde todos os dados são transmitidos para um único ponto de processamento, a IoT demanda arquiteturas distribuídas capazes de lidar com grandes volumes de informações, operar com latências reduzidas e manter um desempenho consistente mesmo em cenários de conectividade instável. Nesse contexto, a computação distribuída em IoT se consolida como uma estratégia essencial, permitindo que o processamento ocorra em diferentes pontos da rede, com variados níveis de autonomia e capacidade computacional.

A diversidade dos dispositivos utilizados em soluções IoT, que podem incluir sensores básicos, microcontroladores, gateways, servidores locais e plataformas em nuvem, exige modelos escaláveis e flexíveis. Nos últimos anos, essa diversidade impulsionou o surgimento de arquiteturas com múltiplas camadas de processamento, cada uma desempenhando um papel específico no ciclo de vida dos dados. Essa descentralização da lógica computacional possibilita respostas mais rápidas a eventos críticos, aumenta a resiliência em ambientes sujeitos a falhas e promove uma gestão mais eficiente do consumo energético.

De forma geral, a computação distribuída em IoT é organizada em três níveis principais: **borda** (edge), **névoa** (fog) e **nuvem** (cloud). Essa divisão representa não apenas a localização física onde o processamento ocorre, mas também a complexidade das tarefas atribuídas a cada nível. A **borda** executa operações imediatas, como a leitura e a filtragem de dados de sensores. A **névoa** — posicionada entre a borda e a nuvem — possibilita análises locais mais avançadas, incluindo detecção de padrões e agregações de dados. Por último, a **nuvem** concentra o armazenamento histórico, a análise de grandes volumes de informações, a visualização de métricas complexas e a integração com recursos como Inteligência Artificial, Aprendizado de Máquina e Bancos de Dados Especializados.

Essa hierarquia torna possível responder a eventos de maneira eficiente e otimizada. Em um sistema de monitoramento sísmico, por exemplo, a borda poderia

detectar anomalias no sinal de um acelerômetro, a névoa consolidaria dados de sensores de uma região e acionaria alertas locais, enquanto a nuvem armazenaria o histórico para análises detalhadas e cruzamento com variáveis ambientais. Dessa forma, cada camada cumpre uma função específica e complementar.

A adaptabilidade também é um fator relevante. Em ambientes urbanos com conectividade estável, a nuvem pode desempenhar o papel central no sistema, recebendo e processando dados de forma contínua. Já em áreas remotas ou com infraestrutura limitada, é fundamental que decisões sejam tomadas localmente, ampliando a importância da borda e da névoa para garantir autonomia e continuidade operacional. Essa versatilidade torna a computação distribuída uma escolha estratégica em setores como agricultura de precisão, saúde conectada, cidades inteligentes e processos industriais com requisitos de tempo real.

No entanto, a adoção dessa abordagem traz alguns desafios como a necessidade de sincronização entre diferentes camadas, a proteção dos dados em múltiplos pontos da rede e a otimização do firmware embarcado. O projetista de soluções IoT deve estar preparado para avaliar não apenas as tecnologias disponíveis, mas também as condições operacionais do ambiente, equilibrando custo, desempenho e segurança.

Compreender essa estrutura distribuída é o ponto de partida para entender o papel da computação em nuvem, que se destaca como elemento central em muitas arquiteturas IoT modernas. Na próxima seção, serão apresentados os conceitos fundamentais dessa abordagem, explorando suas características, possibilidades e o potencial que oferece para aplicações conectadas em larga escala.

1.2 Conceito de Cloud, Fog e Edge Computing

Compreender os conceitos de cloud computing, fog computing e edge computing é fundamental para projetar arquiteturas eficientes e escaláveis em sistemas de IoT. Essas três abordagens representam diferentes pontos de processamento dos dados, cada uma com características próprias quanto à latência, capacidade de processamento, consumo energético, segurança e complexidade de implementação. A escolha da estratégia mais adequada depende diretamente dos

requisitos da aplicação, da infraestrutura disponível e do nível de autonomia esperado para os dispositivos no ambiente de operação.

A computação em nuvem é caracterizada pelo processamento e armazenamento de dados em servidores remotos, normalmente distribuídos geograficamente e acessados por meio da internet. Nessa abordagem, os dispositivos IoT atuam como coletores de dados, enviando as informações para a nuvem, onde ocorre o processamento centralizado, a análise estatística, a criação de dashboards e a aplicação de algoritmos avançados de Inteligência Artificial. As plataformas em nuvem oferecem recursos praticamente ilimitados de escalabilidade, além de integração com APIs e serviços de terceiros. Essa capacidade a torna ideal para cenários que exigem armazenamento de longo prazo, cruzamento de dados de múltiplas fontes e análises complexas. No entanto, o uso da nuvem implica em maior dependência de conectividade estável, latência mais elevada e necessidade de medidas adicionais para proteção de dados sensíveis.

A computação em névoa atua como camada intermediária entre os dispositivos de borda e a nuvem. Nesse modelo, o processamento é realizado em equipamentos locais, como gateways, roteadores inteligentes, computadores embarcados ou servidores próximos ao ambiente de coleta. A névoa possibilita respostas mais rápidas ao reduzir a distância que os dados percorrem até serem analisados, além de diminuir o tráfego de rede e aliviar a carga sobre os servidores em nuvem. Essa abordagem é indicada para ambientes industriais, agrícolas e urbanos em que decisões automatizadas precisam ser tomadas com agilidade e, ao mesmo tempo, mantendo a integração com sistemas de maior capacidade de processamento e armazenamento.

Embora distintas, essas abordagens podem ser combinadas para formar arquiteturas híbridas, aproveitando os pontos fortes de cada nível. Em soluções modernas de IoT, é comum que a borda execute decisões rápidas, a névoa consolide informações e a nuvem concentre análises e armazenamento de longo prazo. No contexto deste capítulo, o foco será aprofundar o papel da computação em nuvem e demonstrar como sua aplicação pode potencializar projetos com microcontroladores ESP32 e sensores embarcados.

1.3 Arquiteturas distribuídas em sistemas ciberfísicos

Os sistemas ciberfísicos representam a integração entre o mundo físico e o digital, combinando sensores, atuadores, microcontroladores, redes de comunicação e plataformas computacionais em uma estrutura capaz de perceber, processar e reagir ao ambiente de forma inteligente. Em soluções baseadas em Internet das Coisas, o desenho da arquitetura distribuída é essencial para garantir escalabilidade, eficiência energética, confiabilidade e autonomia nas respostas aos eventos do mundo real.

A arquitetura distribuída desses sistemas baseia-se no princípio de descentralizar o processamento, a coleta e a tomada de decisão. Em vez de concentrar toda a inteligência em um único ponto — como em um servidor central —, as responsabilidades são distribuídas por diferentes camadas e dispositivos. Isso permite uma resposta mais rápida e resiliente a variações ambientais, ao mesmo tempo em que possibilita que cada camada opere de acordo com suas capacidades.

A próxima figura apresenta um modelo clássico de arquitetura distribuída em IoT, destacando os níveis de sensoriamento, comunicação, pré-processamento, análise e visualização, com a clara divisão das funções entre as camadas de borda, névoa e nuvem. Essa organização hierárquica facilita a compreensão do papel de cada nível e reforça a importância da coordenação entre eles.

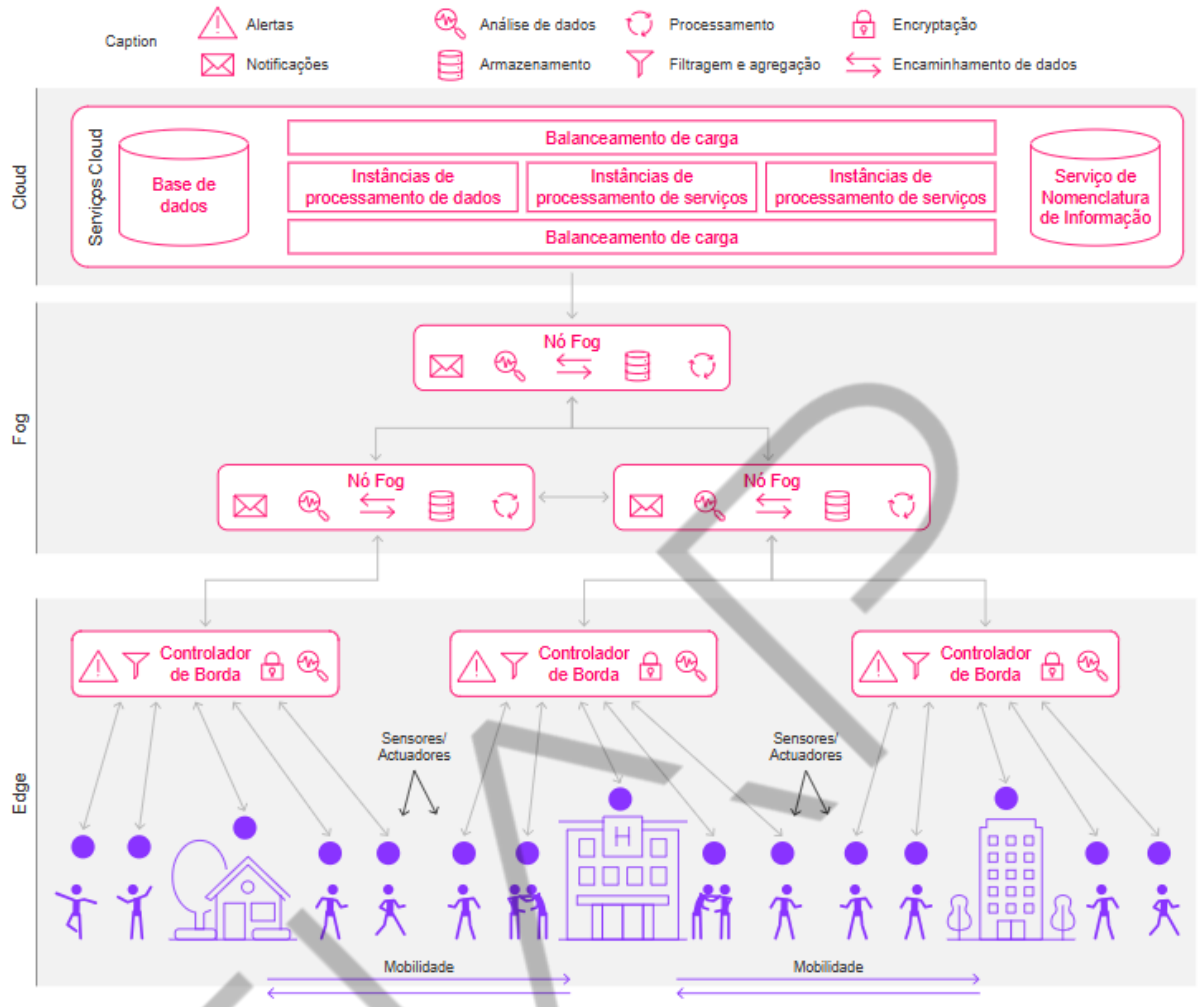


Figura 1 - Arquitetura distribuída em sistemas ciberfísicos com camadas de borda, névoa e nuvem
Fonte: Fonseca *et al.* (2022), adaptado por FIAP (2025)

A camada de borda é composta por sensores e microcontroladores responsáveis pela aquisição de dados e pela execução de lógica embarcada. Em muitos casos, essa lógica utiliza modelos de Inteligência Artificial treinados para identificar eventos relevantes, classificar padrões ou tomar decisões de forma autônoma com base nas leituras dos sensores. A camada intermediária, correspondente à névoa, agrega dados de múltiplos dispositivos, aplica filtros, realiza compressão e detecta padrões em nível local ou regional. Por fim, a camada de nuvem concentra o armazenamento histórico, a integração com serviços externos, a execução de algoritmos de IA em larga escala e a disponibilização de dashboards para visualização e análise estratégica.

Essa estrutura modular oferece benefícios operacionais claros. Ao mesmo tempo em que a borda e a névoa permitem decisões rápidas e independentes, a

nuvem garante uma visão global do sistema, possibilitando análises preditivas, cruzamento de dados de diversas fontes e integração com serviços corporativos ou de Inteligência Artificial. Em muitas aplicações, a nuvem atua como elemento de consolidação e de coordenação, sendo fundamental para gerar insights de longo prazo e apoiar a tomada de decisões estratégicas.

O uso de arquiteturas distribuídas é indispensável em aplicações críticas como veículos autônomos, linhas de produção automatizadas, hospitais inteligentes e sistemas de resposta a desastres. Nessas situações, é necessário garantir tanto a agilidade local quanto a capacidade de análise centralizada. O projetista de sistemas ciberfísicos deve, portanto, definir cuidadosamente quais funções atribuir a cada camada e como garantir a interoperabilidade entre elas, mantendo um equilíbrio entre desempenho, custo e segurança.

Compreender essa organização é fundamental para aproveitar ao máximo o potencial da nuvem. Na próxima seção, serão exploradas as vantagens e limitações dessa abordagem, evidenciando como o seu uso pode ampliar a capacidade de sistemas IoT e sustentar projetos escaláveis com alto valor estratégico.

1.4 Vantagens e desvantagens da computação em nuvem

Ao avaliar a adoção da computação em nuvem em projetos de Internet das Coisas, é importante compreender seus pontos fortes e suas limitações, considerando o tipo de aplicação, os requisitos de desempenho, a infraestrutura disponível e os objetivos estratégicos do sistema. Essa análise permite que o projetista faça escolhas mais adequadas e dimensione corretamente a arquitetura, explorando ao máximo os benefícios que a nuvem pode oferecer.

A computação em nuvem se destaca por sua alta escalabilidade, capacidade de armazenamento praticamente ilimitada e facilidade de integração com serviços avançados, como algoritmos de aprendizado de máquina, dashboards interativos e sistemas corporativos. Essa abordagem é especialmente vantajosa para aplicações que demandam análise preditiva, tratamento de grandes volumes de dados históricos e consolidação de informações provenientes de múltiplas fontes.

Outro diferencial da nuvem é a possibilidade de acesso remoto seguro e centralizado, permitindo que equipes distribuídas geograficamente visualizem e

analisem dados em tempo real. Além disso, os provedores de serviços em nuvem oferecem infraestrutura de alta disponibilidade, backups automáticos e recursos de segurança integrados, reduzindo a necessidade de investimentos elevados em infraestrutura própria.

No entanto, a nuvem também apresenta desafios que precisam ser considerados. A dependência de conectividade constante pode comprometer o desempenho de aplicações críticas em locais com acesso limitado à internet. A latência inerente à transmissão de dados para servidores remotos pode impactar sistemas que exigem resposta imediata. Questões relacionadas à privacidade e à conformidade regulatória também merecem atenção, principalmente em setores sensíveis como saúde e financeiro.

O quadro a seguir apresenta um resumo das principais vantagens e desvantagens da computação em nuvem no contexto de IoT, servindo como referência para a tomada de decisão.

Vantagens	Desvantagens
Alta escalabilidade e capacidade de armazenamento.	Dependência de conectividade constante.
Integração com Inteligência Artificial e serviços analíticos avançados.	Latência maior em comparação a arquiteturas locais.
Acesso remoto centralizado e seguro.	Possíveis riscos à privacidade e conformidade regulatória.
Alta disponibilidade e redundância oferecidas por provedores.	Custos recorrentes de operação e armazenamento.

Quadro 1 - Vantagens e desvantagens da computação em nuvem
Fonte: Elaborado pelo autor (2025)

Ao compreender essas características, o projetista pode decidir de forma estratégica quando utilizar a nuvem como elemento central de sua arquitetura IoT, seja de forma isolada ou combinada com outras camadas de processamento. Na próxima subseção, serão apresentados casos de uso típicos que ilustram como a computação em nuvem pode agregar valor e potencializar o desempenho de sistemas conectados.

1.4.1 Casos de uso típicos em IoT

A aplicação das arquiteturas de cloud, fog e edge computing é amplamente observada em diversos setores da indústria, agricultura, saúde, cidades inteligentes, logística, entre outros. Cada abordagem é escolhida com base nos requisitos de latência, capacidade de processamento, disponibilidade de conexão e nível de criticidade das decisões.

Na indústria 4.0, por exemplo, a computação em borda é utilizada para monitoramento de máquinas em tempo real, com sensores que executam lógica embarcada capaz de detectar anomalias e acionar paradas automáticas em casos de superaquecimento ou vibrações fora do padrão. Essa resposta imediata só é possível com o processamento local, sem depender da comunicação com servidores externos.

Na agricultura de precisão, sensores espalhados pela lavoura coletam dados de umidade do solo, temperatura, exposição solar e outros fatores ambientais. Gateways locais aplicam lógicas de controle baseadas em fog computing, determinando automaticamente a irrigação em cada região da plantação. Esses dados também podem ser enviados à nuvem para registro histórico, cruzamento com imagens de satélite e análises preditivas para planejamento de safra.

Em sistemas de saúde conectada, os dispositivos vestíveis (*wearables*) são exemplos clássicos de edge computing, pois realizam medições constantes de batimentos cardíacos, temperatura corporal, oxigenação do sangue e outros sinais vitais, processando esses dados localmente e emitindo alertas ao usuário em caso de risco. Hospitais, por sua vez, podem utilizar fog computing para integrar e analisar dados de vários pacientes em tempo real, otimizando a gestão de leitos, medicamentos e equipe médica. Já os sistemas de cloud são empregados no armazenamento de históricos médicos e aplicações de apoio à decisão clínica baseadas em Inteligência Artificial.

Cidades inteligentes também representam um ecossistema que se beneficia de todas as abordagens. Sistemas de trânsito podem usar sensores locais (edge) para detectar congestionamentos, semáforos inteligentes podem se adaptar em tempo real com apoio de fog computing, enquanto painéis de controle centralizados na prefeitura utilizam cloud computing para analisar padrões de mobilidade e planejar obras viárias.

No setor logístico, a combinação das três abordagens é comum. Veículos com sensores de localização e temperatura utilizam edge computing para acionar alarmes locais em caso de violação da carga. Centros de distribuição operam com fog computing para reconfiguração dinâmica de rotas, e a análise de desempenho logístico é feita em plataformas de cloud.

Esses exemplos mostram que a adoção de uma abordagem não exclui as demais. Sistemas bem projetados costumam utilizar modelos híbridos, combinando as três arquiteturas para alcançar escalabilidade, eficiência, segurança e respostas rápidas. Entre essas possibilidades, a computação em nuvem se destaca por permitir integrações avançadas e visualização de dados em tempo real a partir de qualquer lugar, tornando-se uma base sólida para a criação de soluções IoT conectadas e analíticas. Na próxima seção, será apresentada a aplicação prática desse conceito, utilizando microcontroladores e plataformas de monitoramento remoto, para demonstrar como essas vantagens se manifestam em cenários reais, desde a captura de dados até sua análise visual.

FIAP

2 CLOUD COMPUTING COM ESP32 E GRAFANA CLOUD

A adoção da arquitetura em nuvem em projetos de Internet das Coisas tem viabilizado soluções escaláveis, conectadas e com forte capacidade de análise remota. Dispositivos embarcados podem operar com consumo energético otimizado, transmitindo apenas os dados coletados para plataformas robustas de armazenamento e visualização, aproveitando o poder computacional remoto.

Essa abordagem se mostra especialmente útil em aplicações que exigem coleta contínua de dados, histórico de medições, acesso remoto por múltiplos usuários e integração com painéis interativos. Em projetos educacionais e protótipos de baixo custo, é possível aproveitar serviços gratuitos na nuvem para experimentar arquiteturas completas, utilizando ferramentas como HiveMQ Cloud para comunicação MQTT, InfluxDB Cloud para banco de dados temporal e Grafana Cloud para visualização em tempo real.

A experiência prática apresentada nas próximas subseções exemplifica uma arquitetura funcional baseada em nuvem, com envio de dados de sensores por meio de um microcontrolador ESP32. Essa implementação demonstra como cada componente da infraestrutura se conecta desde a transmissão até o armazenamento e visualização, permitindo ao aluno vivenciar uma solução real de IoT com tecnologia amplamente utilizada no mercado.

Também serão discutidas alternativas e variações que ampliam a aplicabilidade da arquitetura em nuvem, considerando diferentes tipos de sensores, protocolos, provedores e integrações com sistemas corporativos ou de análise inteligente.

2.1 Fundamentos da computação em nuvem aplicada à IoT

A computação em nuvem desempenha um papel central em muitos sistemas de Internet das Coisas, atuando como o ponto de convergência para o armazenamento, o processamento e a análise de dados provenientes de dispositivos embarcados espalhados geograficamente. Essa abordagem permite que sensores e microcontroladores de baixo custo, como o ESP32, se conectem a serviços remotos altamente escaláveis, acessando uma gama de funcionalidades que dificilmente seriam suportadas localmente devido às limitações de hardware.

Ao empregar a nuvem como parte da arquitetura IoT, é possível delegar tarefas complexas como a persistência de grandes volumes de dados, o treinamento e inferência de modelos de aprendizado de máquina, a geração de relatórios e visualizações dinâmicas, além da integração com sistemas corporativos e serviços externos. Isso torna a computação em nuvem particularmente vantajosa em aplicações que exigem análise histórica, padronização dos dados, correlação entre múltiplas fontes e monitoramento centralizado.

Na prática, uma solução baseada em nuvem geralmente segue um fluxo lógico que se inicia com a coleta de dados no dispositivo embarcado, seu envio por meio de protocolos de comunicação leve (como o MQTT), o armazenamento em bancos de dados temporais otimizados para séries temporais (como o InfluxDB Cloud) e, por fim, a visualização dos dados em dashboards interativos hospedados em plataformas como o Grafana Cloud. Todo esse ecossistema é orquestrado em tempo real, permitindo que engenheiros e tomadores de decisão acompanhem o funcionamento do sistema à distância, em qualquer lugar com acesso à internet.

A figura a seguir ilustra uma arquitetura simplificada de computação em nuvem aplicada à IoT, evidenciando a separação entre os dispositivos físicos, o transporte dos dados e os serviços oferecidos em nuvem. Essa divisão de responsabilidades torna o sistema mais modular, flexível e escalável, adaptando-se bem a diferentes contextos de aplicação.

FIAP

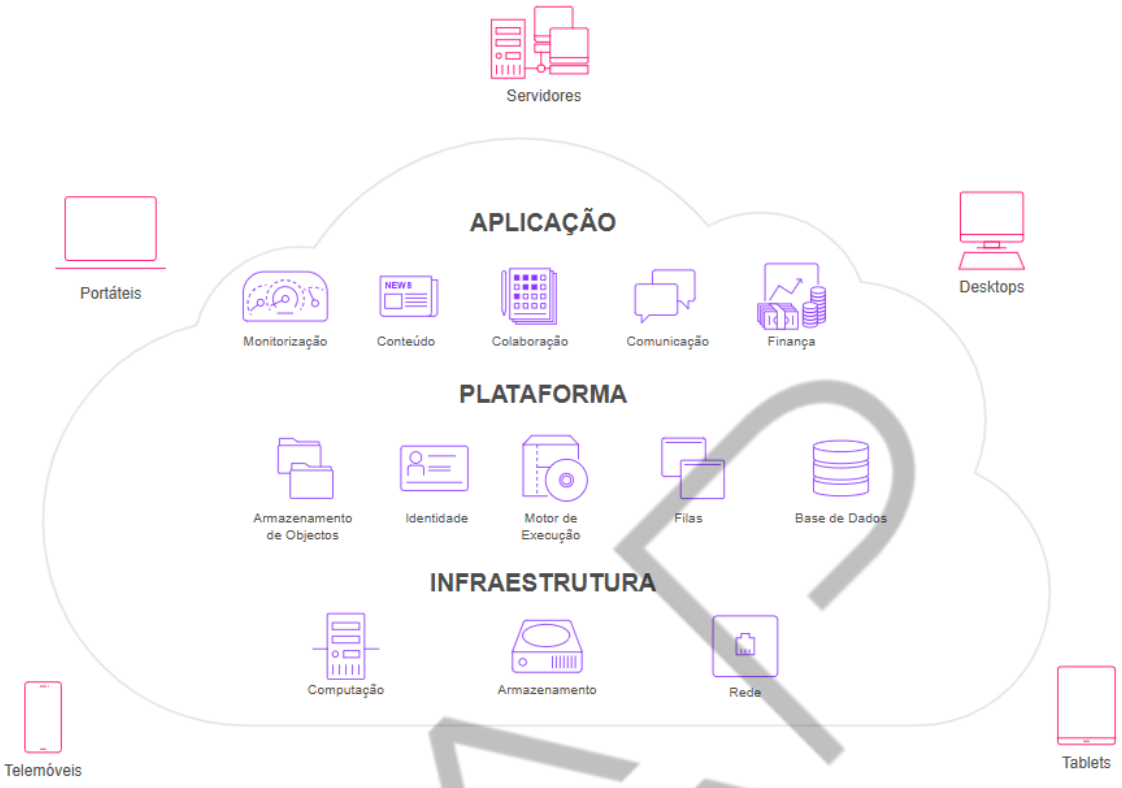


Figura 2 - Arquitetura de computação em nuvem com camadas de infraestrutura, plataforma e aplicação em IoT
Fonte: Cruz, A.; Santos, R. (2015), adaptado pelo autor (2025)

2.2 Experiência prática com ESP32 e visualização em nuvem

A prática descrita nesta seção demonstra a aplicação de uma arquitetura de computação em nuvem com um sistema embarcado baseado em ESP32. O objetivo é transmitir dados de aceleração coletados pelo sensor MPU6050, processá-los e visualizá-los remotamente por meio de uma plataforma conectada.

A próxima figura ilustra o fluxo de dados da experiência, destacando os principais componentes envolvidos na comunicação, processamento, armazenamento e visualização das informações.

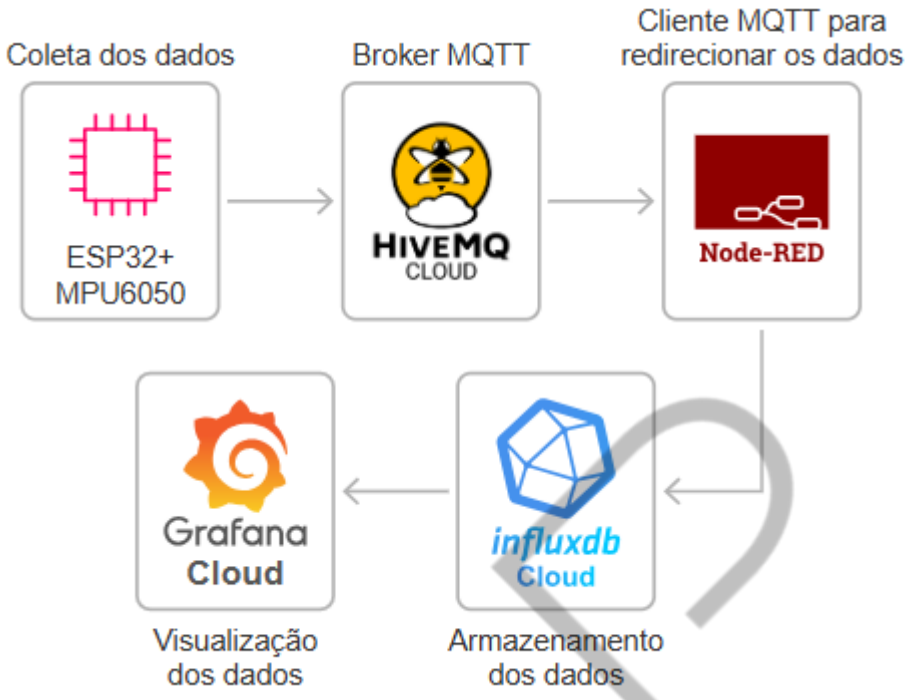


Figura 3 - Arquitetura da experiência de computação em nuvem com ESP32 e plataformas integradas
Fonte: Elaborado pelo autor (2025)

O processo se inicia com a coleta dos dados no dispositivo embarcado (ESP32 + MPU6050), que envia os dados via protocolo MQTT para o broker HiveMQ Cloud. Em seguida, o Node-RED atua como cliente MQTT, recebendo as mensagens e encaminhando os dados estruturados para o InfluxDB Cloud. Esse banco de dados é otimizado para armazenar séries temporais e permite a consulta eficiente dos dados capturados. Por fim, o Grafana Cloud se conecta diretamente ao InfluxDB Cloud para criar dashboards interativos e dinâmicos, facilitando a análise remota das informações.

A seguir, as próximas subseções detalham cada etapa da implementação, desde a configuração do hardware até a montagem dos gráficos no ambiente em nuvem. Esse processo não apenas reforça os conceitos de cloud computing na prática, como também demonstra a integração fluida entre dispositivos IoT e plataformas distribuídas.

2.2.1 Hardware – ESP32 e MPU6050

Nesta experiência, foi utilizada uma arquitetura de hardware composta pelo microcontrolador ESP32 DevKit V4 e pelo sensor inercial MPU6050. O ESP32 é um

microcontrolador de 32 bits com conectividade Wi-Fi e Bluetooth integrada, projetado para aplicações de IoT que exigem comunicação sem fio, processamento local e baixo consumo de energia. Sua versatilidade o torna ideal para projetos que envolvem o envio de dados para servidores remotos ou plataformas de visualização na nuvem.

O sensor MPU6050 é um dispositivo MEMS (Micro-Electro-Mechanical System) que integra um acelerômetro e um giroscópio, ambos de três eixos. Ele é amplamente utilizado para medir aceleração linear e rotação angular em aplicações embarcadas. Além disso, possui um DMP (Digital Motion Processor) interno, capaz de realizar fusão sensorial e fornecer orientação espacial precisa, embora esse recurso não tenha sido explorado nesta experiência.

A comunicação entre o ESP32 e o MPU6050 é realizada por meio do barramento I²C, utilizando os pinos GPIO 21 (SDA) e GPIO 22 (SCL) do ESP32. O sensor é alimentado com 3,3 V e utiliza um GND comum com o microcontrolador. Essa configuração permite realizar leituras contínuas da aceleração nos três eixos e calcular a magnitude do vetor de aceleração com alta frequência.

A figura ilustra o diagrama de conexão entre os dois dispositivos, simulado no ambiente virtual Wokwi:

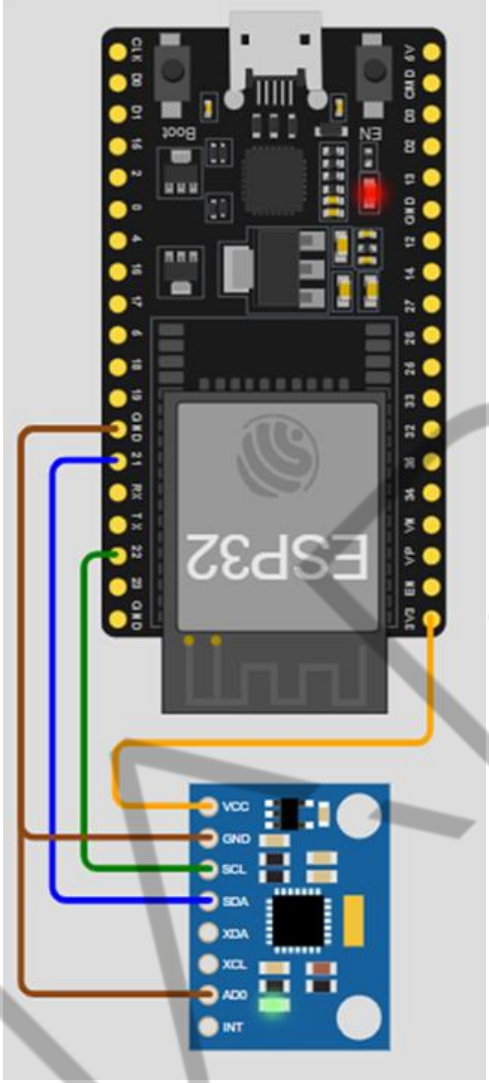


Figura 4 - Diagrama de conexão entre ESP32 DevKit V4 e MPU6050 via I²C
Fonte: WOKWI (2025)

A seguir, apresenta-se o funcionamento completo do código desenvolvido para esta experiência, dividido em quatro partes: definição de bibliotecas e parâmetros; conexão à nuvem; leitura dos dados do sensor; e envio dos dados formatados via MQTT.

A disciplina de PERC foi pensada para aproximar você da realidade prática da eletrônica aplicada à IA. Por isso, os exemplos de códigos e sensores abordados neste e nos próximos capítulos não estão disponíveis em simuladores computacionais, e sim, em placas reais. A escolha de trabalhar com placas e sensores reais ajuda a conectar a teoria com a prática do mundo físico, mostrando como os sistemas realmente funcionam no dia a dia. Não é obrigatório adquirir componentes para acompanhar a disciplina, mas caso você opte por comprar alguns sensores e montar pequenos circuitos de IoT, a sua experiência será muito mais imersiva e rica de conhecimento.

Neste vídeo, mostramos como realizar a migração do Wokwi.com — um simulador eletrônico online com alguns sensores — para o VS Code no seu computador. Assim, você poderá rodar seus projetos de forma local, sem limitações de desempenho e sem custos adicionais com assinaturas.

Para acompanhar o passo a passo, será necessário utilizar os arquivos platformio.ini e wokwi.toml, que estão disponíveis na pasta Assets. Esses arquivos já estão configurados para facilitar sua transição e garantir que o ambiente funcione corretamente.

Cada parte está descrita com base na lógica do fluxo de cloud computing abordado na seção anterior.

O código-fonte “Inclusão de bibliotecas e definição de parâmetros” mostra as bibliotecas utilizadas e a definição dos parâmetros essenciais de conexão com o sensor e com o broker MQTT na nuvem. Esse bloco realiza a importação das bibliotecas necessárias para comunicação sem fio, protocolo MQTT e leitura do sensor via I²C. Em seguida, define o endereço I²C do MPU6050, configura o cliente MQTT com segurança TLS e estabelece as credenciais de rede Wi-Fi e do broker HiveMQ Cloud.

```
    #include <WiFi.h>
    #include <WiFiClientSecure.h>
    #include <PubSubClient.h>
    #include <Wire.h>
    #include <math.h>

    #define MPU_ADDR 0x68

    WiFiClientSecure espClient;
    PubSubClient client(espClient);

    const char* ssid = "FIAP-IOT";
    const char* password = "F!@p25.IOT";

    const            char*            mqtt_server            =
"a2314fa38190415199578b60e005d9d2.s1.eu.hivemq.cloud";
    const int mqtt_port = 8883;
    const char* mqtt_user = "FIAP-PROFESSOR";
    const char* mqtt_pass = "Teste1234";
```

Código-fonte 1 - Inclusão de bibliotecas e definição de parâmetros
Fonte: Elaborado pelo autor (2025)

O código-fonte “Conexão à nuvem e inicialização de sensores” realiza a conexão do ESP32 à rede Wi-Fi, ativa o sensor MPU6050 e configura o cliente MQTT

seguro para conexão com o broker na nuvem. Esse trecho inicializa o Monitor Serial, configura a comunicação I²C e conecta o ESP32 à rede Wi-Fi. O sensor MPU6050 é ativado ao ser retirado do modo de suspensão. A função reconnect() assegura que o dispositivo tente se reconectar ao broker MQTT sempre que a conexão for perdida. A comunicação segura é estabelecida com TLS, mesmo sem verificação de certificado.

```
    void reconnect() {
      while (!client.connected()) {
        if      (client.connect("ESP32Client",      mqtt_user,
mqtt_pass)) break;
        delay(5000);
      }
    }

    void setup() {
      Serial.begin(115200);
      Wire.begin();

      WiFi.begin(ssid, password);
      while (WiFi.status() != WL_CONNECTED) delay(500);

      Wire.beginTransmission(MPU_ADDR);
      Wire.write(0x6B);
      Wire.write(0);
      Wire.endTransmission(true);

      espClient.setInsecure();
      client.setServer(mqtt_server, mqtt_port);
    }
```

Código-fonte 2 - Conexão à nuvem e inicialização de sensores
Fonte: Elaborado pelo autor (2025)

O código-fonte “Leitura dos dados do MPU6050” mostra como os dados são lidos do sensor MPU6050 e convertidos para valores em g, além do cálculo da magnitude total do vetor de aceleração. Essa parte do código realiza a leitura dos registros do acelerômetro MPU6050 por meio do protocolo I²C. Os dados brutos dos eixos X, Y e Z são convertidos para valores em “g”. Em seguida, a magnitude total do vetor de aceleração é calculada, representando a força total de movimento atuando sobre o sensor.

```
    Wire.beginTransmission(MPU_ADDR);
    Wire.write(0x3B);
    Wire.endTransmission(false);
    Wire.requestFrom(MPU_ADDR, 6, true);

    int16_t ax_raw = Wire.read() << 8 | Wire.read();
    int16_t ay_raw = Wire.read() << 8 | Wire.read();
    int16_t az_raw = Wire.read() << 8 | Wire.read();

    float ax = ax_raw / 16384.0;
    float ay = ay_raw / 16384.0;
    float az = az_raw / 16384.0;

    float g = sqrt(ax * ax + ay * ay + az * az);
```

Código-fonte 3 - Leitura dos dados do MPU6050
Fonte: Elaborado pelo autor (2025)

Por fim, o código-fonte "Formatação e envio dos dados para a nuvem" exibe como os dados são organizados em uma string no formato JSON com duas casas decimais e enviados ao tópico MQTT sensor/movimento. A publicação é feita utilizando protocolo seguro via TLS com o broker HiveMQ Cloud. A cada segundo, uma nova leitura é enviada, permitindo a visualização contínua em dashboards como Node-RED, InfluxDB e Grafana. Nesse trecho final, os dados são organizados e transmitidos via MQTT, fechando o ciclo de coleta, processamento e envio para a nuvem.

```
    char msg[100];
    snprintf(msg, sizeof(msg),

"{\"ax\":%.2f,\"ay\":%.2f,\"az\":%.2f,\"g\":%.2f}",
             ax, ay, az, g);

    client.publish("sensor/movimento", msg);
    Serial.println(msg);

    delay(1000);
```

Código-fonte 4 - Formatação e envio dos dados para a nuvem
Fonte: Elaborado pelo autor (2025)

2.2.2 Transmissão MQTT – Broker HiveMQ Cloud

Para realizar a transmissão dos dados do sensor MPU6050 conectados ao ESP32, utiliza-se o protocolo MQTT, um dos mais populares em aplicações IoT por sua leveza, eficiência e baixo consumo de banda. O MQTT depende de um broker para intermediar a comunicação entre o dispositivo que publica os dados (ESP32) e

os clientes que consomem esses dados (como o Node-RED ou bancos de dados em nuvem).

Entre os diversos brokers MQTT disponíveis, como Mosquitto, EMQX, VerneMQ e AWS IoT Core, optou-se neste exemplo pelo HiveMQ Cloud, que oferece uma versão gratuita com suporte a TLS e interface de gerenciamento intuitiva via navegador. A escolha por esse broker se deve à sua facilidade de uso em ambientes educacionais e protótipos rápidos, mas vale destacar que a substituição por outro broker é perfeitamente possível e compatível com a mesma estrutura adotada.

A próxima figura ilustra a tela principal do HiveMQ Cloud logo após a criação de um cluster gratuito. Nela, pode-se visualizar o endereço do broker, a porta segura (TLS) e o status do serviço.

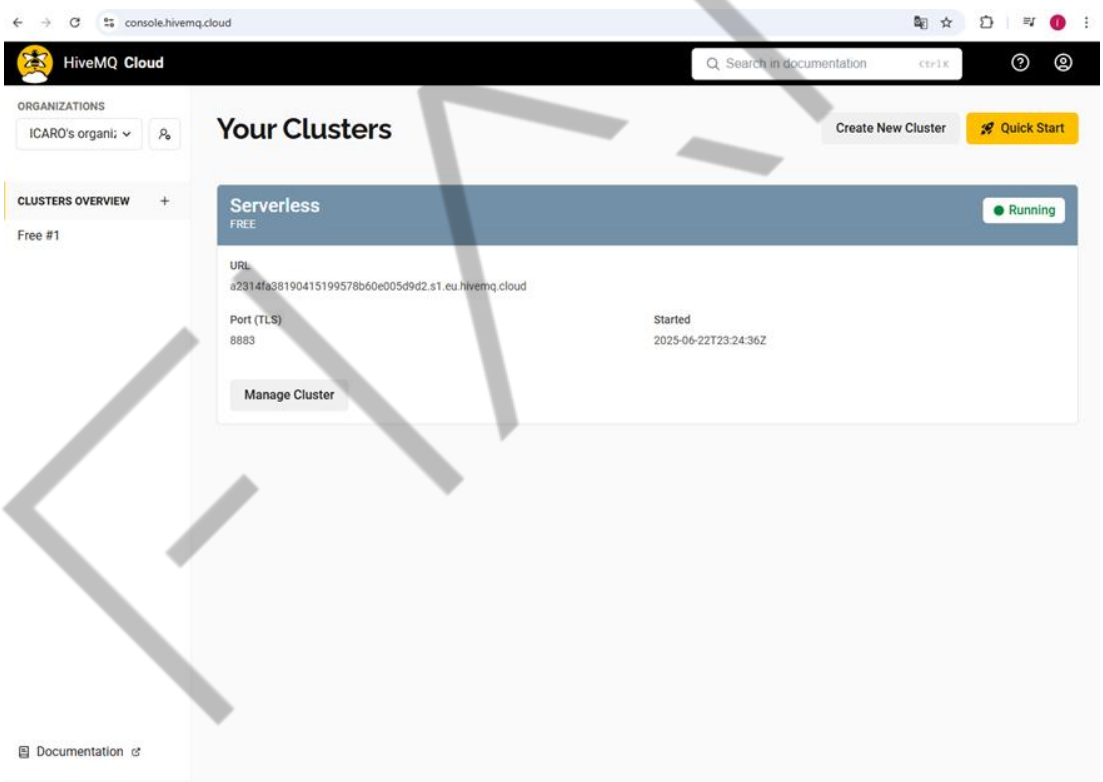


Figura 5 - Tela inicial do HiveMQ Cloud com cluster gratuito configurado
Fonte: HIVEMQ (2025)

Em seguida, é necessário configurar as credenciais de acesso ao cluster. A aba Access Management permite a criação de um usuário (neste caso, “FIAP-PROFESSOR”) com permissões de publicação e subscrição. Isso garante que o ESP32 possa enviar os dados ao tópico desejado com segurança.

A próxima figura mostra essa etapa de configuração:

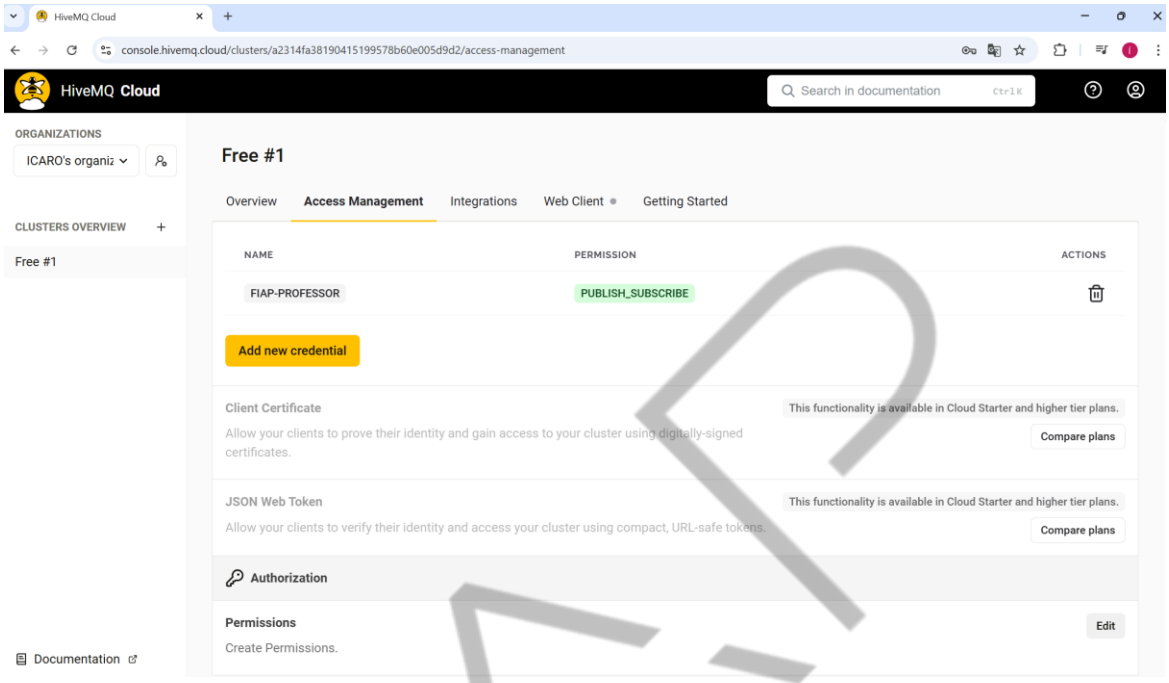


Figura 6 - Definição de credenciais MQTT no painel de gerenciamento do HiveMQ Cloud
Fonte: HIVEMQ (2025)

Com a configuração adequada do HiveMQ Cloud como broker MQTT, o ESP32 está apto a transmitir dados de forma segura e contínua para a nuvem. Essa etapa garante que os dados coletados pelo sensor cheguem ao ambiente de processamento com integridade. A próxima subseção aborda a utilização do Node-RED como ferramenta intermediária para tratamento e redirecionamento desses dados ao sistema de armazenamento em nuvem.

2.2.3 Processamento e encaminhamento – Node-RED

Após a publicação dos dados de movimento pelo ESP32 no broker HiveMQ Cloud, é necessário um ambiente intermediário para processar essas mensagens e encaminhá-las a uma base de dados para posterior análise. Neste projeto, o ambiente utilizado foi o Node-RED, uma plataforma de desenvolvimento visual orientada a fluxos, amplamente adotada em aplicações de IoT. Seu principal diferencial é permitir a construção de sistemas de automação e integração de dados com blocos visuais conectáveis, o que facilita a prototipação e o monitoramento de sistemas embarcados.

O fluxo construído no Node-RED é composto por cinco nós principais que operam sequencialmente para transformar a mensagem recebida via MQTT em um registro pronto para armazenamento. A figura a seguir apresenta o fluxo completo, que contempla a recepção da mensagem, conversão para JSON, formatação para o protocolo da InfluxDB e envio via HTTP.

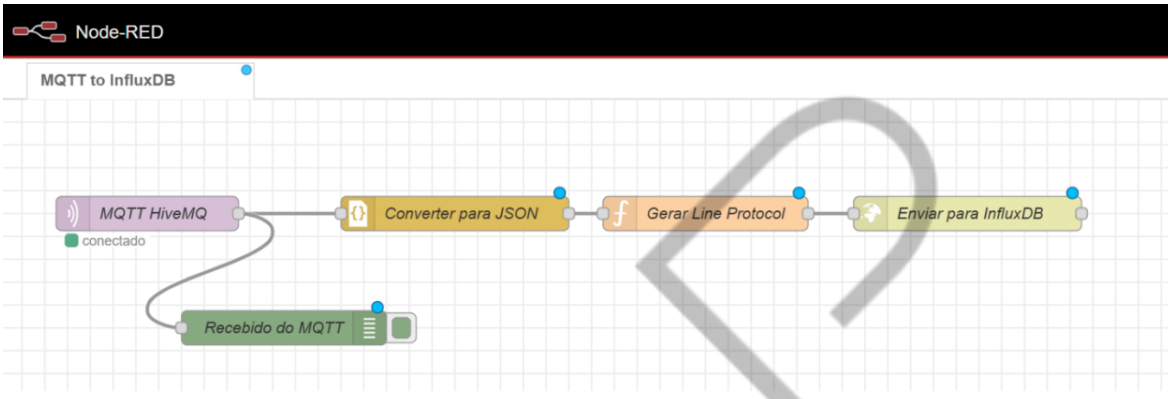


Figura 7 - Configuração do nó MQTT de entrada no Node-RED para conexão com HiveMQ Cloud
Fonte: NODE-RED (2025)

O primeiro nó do fluxo é o MQTT HiveMQ, responsável por se conectar ao broker e assinar o tópico sensor/movimento. Esse nó atua como ponto de entrada dos dados publicados pelo microcontrolador. Ao ser configurado, define-se tanto o tópico a ser escutado quanto as credenciais e parâmetros do servidor remoto. Na configuração mostrada na figura “Configuração dos nós MQTT no Node-RED: tópico e servidor”, observa-se que o servidor MQTT utiliza a porta 8883 com comunicação segura (TLS), além do protocolo MQTT versão 3.1.1. O campo “ID do cliente” é preenchido com o identificador nodered-client, e a opção “sessão limpa” é ativada para garantir que mensagens antigas não sejam mantidas após uma nova conexão.

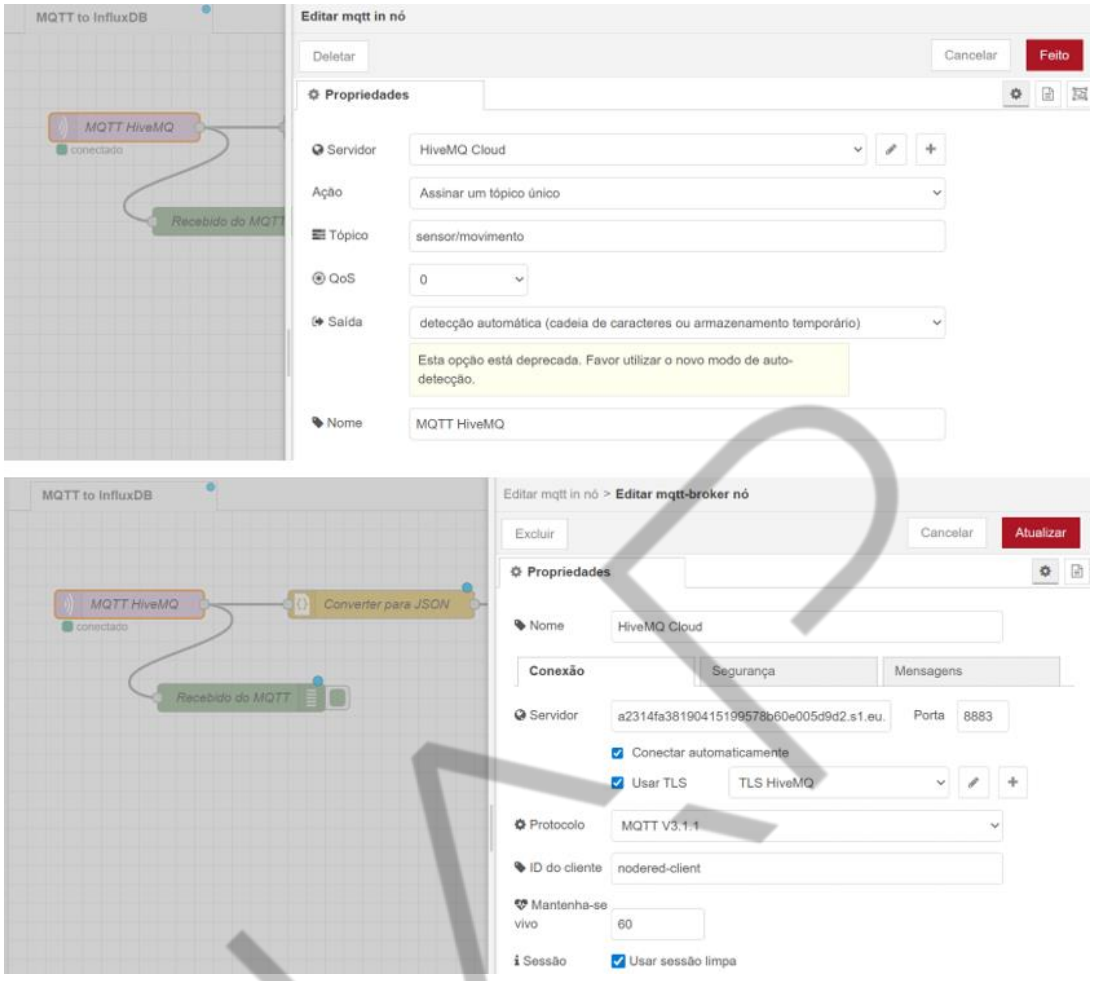


Figura 8 - Configuração dos nós MQTT no Node-RED: tópico e servidor
Fonte: NODE-RED (2025)

Uma vez recebida a mensagem bruta, o nó Converter para JSON transforma a cadeia de caracteres em um objeto estruturado. Isso permite o acesso direto aos campos ax, ay e az, correspondentes aos valores de aceleração nos três eixos. Em seguida, o nó Gerar Line Protocol converte esses dados no formato requerido pela InfluxDB Cloud, conhecido como Line Protocol, utilizado para descrever séries temporais com alta eficiência. Essa transformação é essencial para que o dado seja aceito e interpretado corretamente pela base de dados.

Na etapa final, o nó “Enviar para InfluxDB” realiza uma requisição HTTP do tipo POST à URL da API do InfluxDB Cloud. A configuração desse nó, apresentada na figura “Configuração do nó HTTP para envio dos dados à InfluxDB”, inclui a definição da URL do bucket, o método da requisição e o formato de retorno. A autenticação via token será incluída em um cabeçalho HTTP no próximo passo, o que garante segurança e controle de acesso ao serviço.

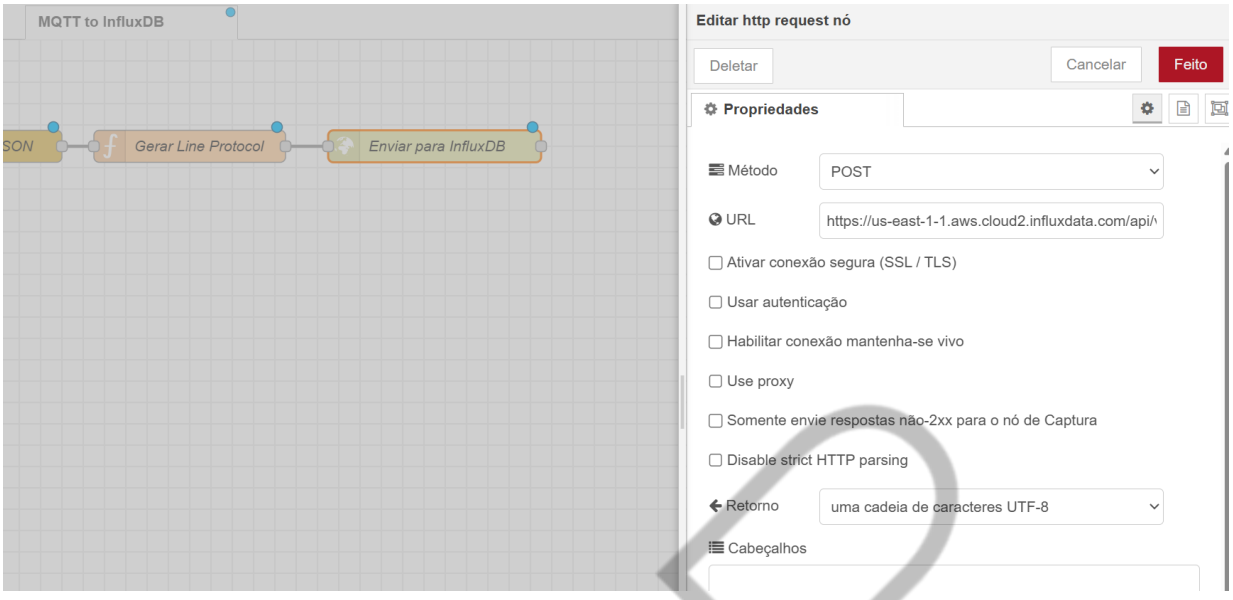


Figura 9 - Configuração do nó HTTP para envio dos dados à InfluxDB
Fonte: NODE-RED (2025)

Com isso, o fluxo do Node-RED atua como um elo entre o ambiente embarcado e a base de dados na nuvem, garantindo que os dados sejam processados e encaminhados corretamente em tempo real. A próxima subseção abordará em detalhes a configuração do serviço InfluxDB Cloud, incluindo a criação de buckets, tokens e validação dos dados recebidos.

2.2.4 Armazenamento de dados – InfluxDB Cloud

Após a etapa de transmissão dos dados via MQTT e o processamento intermediado pelo Node-RED, é necessário definir uma solução confiável para o armazenamento histórico das informações captadas. Existem diversas opções disponíveis no ecossistema de Internet das Coisas, como bancos de dados relacionais, arquivos CSV locais, bases NoSQL e soluções específicas para séries temporais, como o InfluxDB, TimescaleDB, DynamoDB, entre outras.

Neste experimento, optou-se pelo uso do InfluxDB Cloud, uma solução gerenciada e otimizada para dados de séries temporais, ideal para aplicações que envolvem sensores, medições contínuas e análise de tempo real. O InfluxDB Cloud oferece alta disponibilidade, escalabilidade automática e integração nativa com ferramentas como o Grafana.

A primeira etapa foi a criação de um bucket, que funciona como um repositório onde os dados serão armazenados. No projeto, foi criado o bucket Cloud-Computing, com tempo de retenção de 30 dias, o que significa que os dados permanecerão acessíveis por esse período antes de serem automaticamente descartados. A figura a seguir apresenta a tela de gerenciamento do bucket.

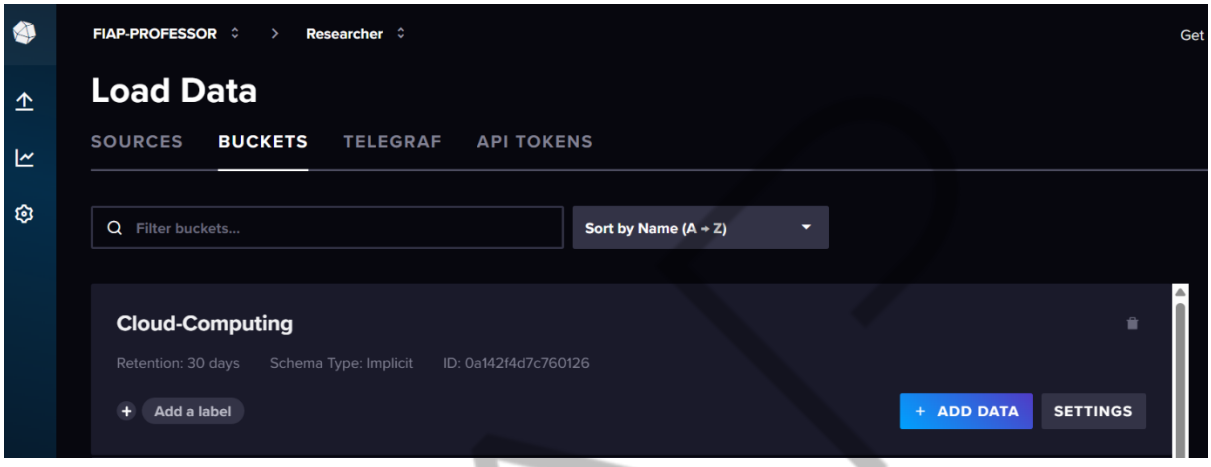


Figura 10 - Bucket “Cloud-Computing” criado no InfluxDB Cloud
Fonte: INFLUXDATA (2025)

Com o bucket configurado, os dados enviados via MQTT e processados pelo Node-RED são encaminhados automaticamente para o InfluxDB. Para visualização e consulta desses dados, a ferramenta disponibiliza o Data Explorer, que permite selecionar o bucket, definir os campos de interesse (como ax, ay, az e g), ajustar o intervalo de tempo e aplicar filtros. Os resultados podem ser exibidos em forma de tabela ou gráfico.

A figura “Consulta dos dados no Data Explorer do InfluxDB Cloud” ilustra o uso do Data Explorer, mostrando os dados do sensor MPU6050 sendo consultados com base nas últimas leituras capturadas.

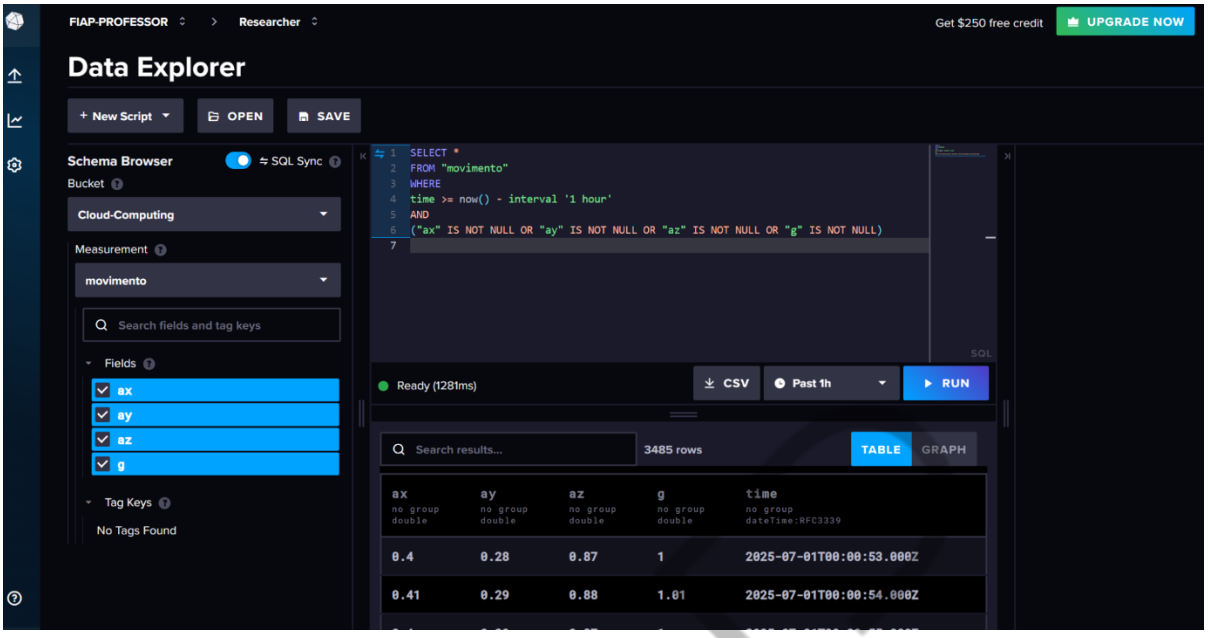


Figura 11 - Consulta dos dados no Data Explorer do InfluxDB Cloud
Fonte: INFLUXDATA (2025)

O InfluxDB Cloud foi escolhido neste exemplo por sua praticidade e foco em dados temporais, mas o conceito de armazenamento pode ser reproduzido com outras plataformas e ferramentas, de acordo com os objetivos, infraestrutura e preferências do projeto. Na próxima subseção, será explorada a criação de painéis dinâmicos com o Grafana Cloud, integrando os dados armazenados para visualização interativa.

2.2.5 Visualização e dashboards – Grafana Cloud

A visualização de dados é uma etapa essencial em aplicações IoT, pois permite interpretar de forma clara as informações transmitidas pelos dispositivos. Entre as diversas ferramentas disponíveis para construção de dashboards interativos, uma das mais utilizadas é o Grafana, especialmente em sua versão Grafana Cloud, que oferece planos gratuitos com recursos robustos.

Neste projeto, o Grafana Cloud foi adotado como exemplo prático para ilustrar a criação de dashboards com base em dados armazenados no InfluxDB Cloud. No entanto, existem outras soluções igualmente viáveis — como ThingsBoard, Kibana, Power BI, Redash, Superset e até mesmo dashboards personalizados com Streamlit ou Dash —, que podem ser escolhidas conforme os requisitos do sistema, custo, facilidade de uso e compatibilidade com o banco de dados.

Após a criação da conta gratuita no Grafana Cloud, o usuário é direcionado à instância da aplicação, onde é possível gerenciar usuários, integrações e fontes de dados. A figura a seguir apresenta a tela inicial do Grafana Cloud, com o menu lateral esquerdo contendo os acessos principais da plataforma.

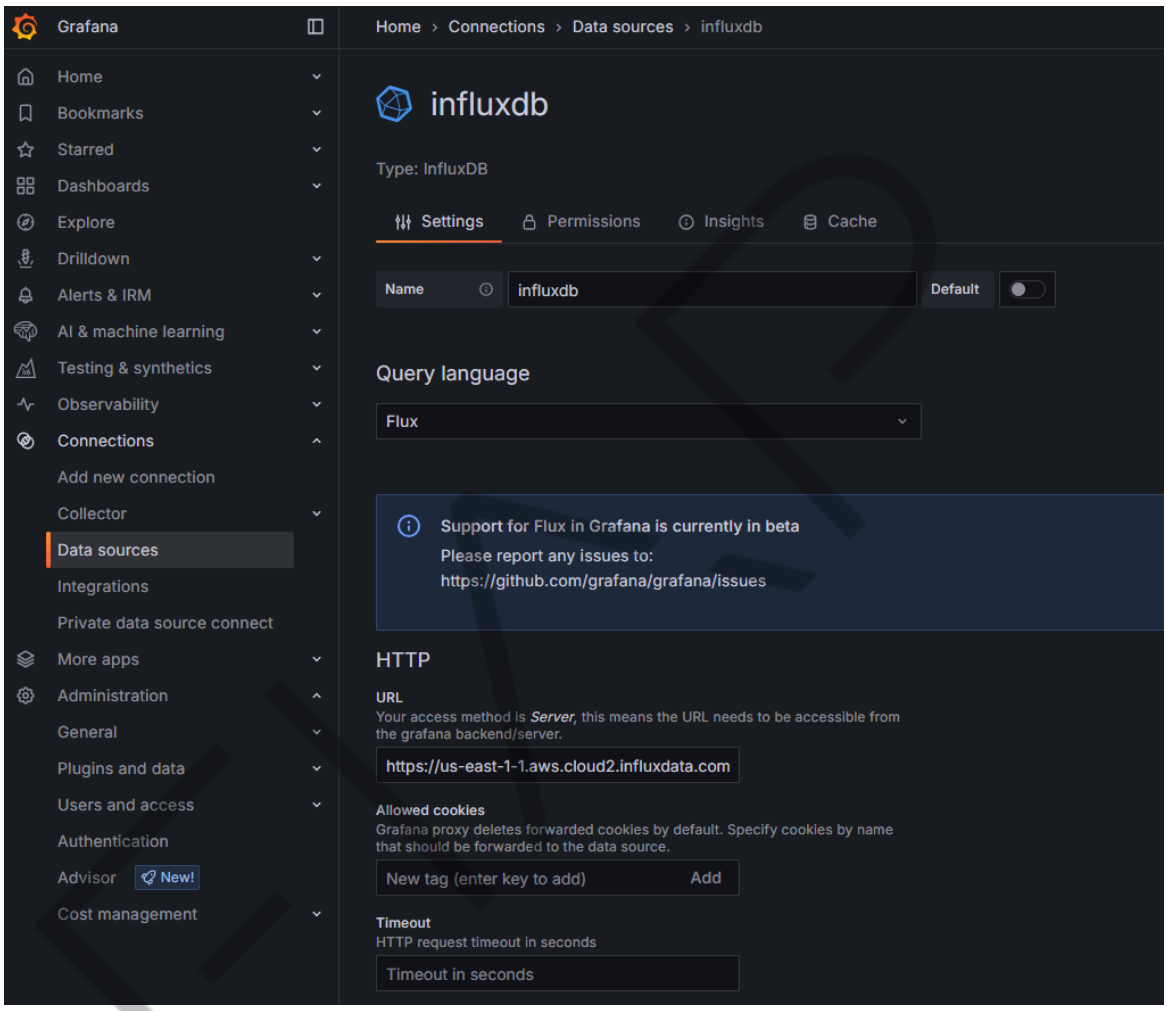


Figura 12 - Configuração da fonte de dados InfluxDB no Grafana Cloud
Fonte: GRAFANA LABS (2025)

A configuração da fonte de dados InfluxDB é realizada no menu Administration, acessível pelo ícone de engrenagem. Em seguida, escolhe-se Data Sources e seleciona-se o InfluxDB. É necessário preencher informações como o bucket, a organização, o token de acesso e a URL da API, todas obtidas no painel do InfluxDB Cloud.

A próxima figura apresenta essa configuração:



Figura 13 - Visualização dos dados de aceleração em tempo real no Grafana Cloud
Fonte: GRAFANA LABS (2025)

Com a fonte de dados conectada, o usuário pode construir dashboards personalizados, criando gráficos baseados em consultas Flux. Além disso, é possível aplicar filtros temporais, realizar agregações, definir cores, criar alertas e organizar os painéis conforme a necessidade da aplicação.

A utilização do Grafana nesta experiência proporciona uma visualização clara e eficaz dos dados sensoriais enviados pelo ESP32. No entanto, reforça-se que a escolha da ferramenta de visualização deve considerar aspectos como escalabilidade, integração com outras plataformas, capacidade de customização e requisitos do projeto.

Com isso, conclui-se a experiência prática de cloud computing, e segue-se agora para a exploração da computação em névoa, onde parte do processamento é descentralizado e executado mais próximo dos dispositivos IoT.

3 COMPARAÇÃO ENTRE NUVEM, NÉVOA E BORDA

A computação em nuvem consolidou-se como um dos pilares das arquiteturas modernas de Internet das Coisas, oferecendo recursos que vão desde o armazenamento massivo de dados até a execução de algoritmos avançados de Inteligência Artificial. Essa capacidade de centralizar e processar informações de maneira escalável permite que dispositivos distribuídos em diferentes localidades contribuam para uma base de dados unificada, facilitando análises complexas e a tomada de decisões estratégicas.

Em sistemas IoT, a nuvem atua como um repositório central para consolidar dados de sensores, câmeras, atuadores e sistemas corporativos, tornando possível a construção de dashboards interativos, a geração de relatórios analíticos e a integração com serviços externos. Além disso, provedores de nuvem oferecem ferramentas que ampliam o potencial das soluções, como APIs de Machine Learning, detecção de anomalias e automação de processos.

Entretanto, o uso exclusivo da nuvem também apresenta restrições. Questões como latência, dependência de conectividade, custos recorrentes e requisitos de conformidade regulatória podem impactar diretamente a viabilidade técnica e financeira de um projeto. Em aplicações críticas, como controle de processos industriais ou sistemas de segurança, a comunicação constante com servidores remotos pode comprometer a capacidade de resposta em tempo real.

Dessa forma, compreender tanto o potencial quanto as limitações da nuvem é essencial para projetar sistemas equilibrados e eficientes. A escolha de como e quando utilizá-la deve considerar não apenas os recursos disponíveis, mas também os objetivos de desempenho, segurança e escalabilidade da solução. Na próxima subseção, será explorado em detalhes o potencial da computação em nuvem e as vantagens que ela oferece para aplicações em IoT.

3.1 Potencial da computação em nuvem

O principal atrativo da computação em nuvem para aplicações em IoT é sua capacidade de oferecer escalabilidade praticamente ilimitada, permitindo que sistemas evoluam de projetos-piloto com poucos dispositivos para implementações massivas com milhares ou milhões de pontos de coleta de dados. Essa flexibilidade

elimina a necessidade de investimentos iniciais elevados em infraestrutura física, uma vez que os recursos de processamento, armazenamento e rede podem ser provisionados sob demanda e ajustados de acordo com as necessidades do projeto.

Outro aspecto relevante é a capacidade de armazenar e gerenciar grandes volumes de dados históricos, o que possibilita análises preditivas, identificação de tendências e detecção de padrões de comportamento. Em um sistema de monitoramento ambiental, por exemplo, dados coletados ao longo de anos podem ser cruzados com informações meteorológicas e modelos climáticos para prever eventos extremos e planejar ações preventivas.

A integração com serviços avançados é outro diferencial. Plataformas de nuvem oferecem APIs e ferramentas nativas para Machine Learning, análise de séries temporais, visão computacional, processamento de linguagem natural e detecção de anomalias. Isso permite que soluções de IoT incorporem Inteligência Artificial de forma simples, sem a necessidade de manter uma infraestrutura local de alto desempenho.

Além disso, a nuvem facilita a criação de dashboards interativos e acessíveis de qualquer lugar do mundo, permitindo que gestores e operadores acompanhem o desempenho do sistema em tempo real. Essa característica é essencial para aplicações que exigem monitoramento contínuo, como redes de sensores industriais, sistemas de transporte inteligente ou monitoramento de saúde de pacientes.

Outro ponto importante é a capacidade de centralizar dados provenientes de múltiplas fontes e tecnologias. Em projetos complexos, é comum que diferentes sensores, protocolos e fabricantes coexistam. A nuvem atua como um hub unificador, permitindo que as informações sejam consolidadas e tratadas de forma padronizada, o que simplifica a integração e o desenvolvimento de novas funcionalidades.

Por fim, a disponibilidade global dos serviços de nuvem garante que equipes distribuídas geograficamente possam colaborar no desenvolvimento, manutenção e operação do sistema, acelerando ciclos de inovação e reduzindo o tempo de resposta a incidentes.

Na próxima subseção, serão analisadas as principais limitações e desafios associados ao uso da nuvem em projetos de IoT, destacando os pontos que exigem atenção no planejamento e na operação dessas soluções.

3.2 Quais são as limitações e desafios da computação em nuvem?

Apesar de seu vasto potencial, a computação em nuvem apresenta restrições e riscos que precisam ser cuidadosamente avaliados no contexto de sistemas IoT. O primeiro desafio é a dependência de conectividade contínua e estável. Em ambientes remotos ou sujeitos a instabilidades de rede, a perda de conexão pode interromper a coleta de dados em tempo real e comprometer a execução de tarefas críticas. Isso é especialmente problemático em aplicações sensíveis, como controle industrial ou monitoramento médico, onde atrasos podem gerar consequências graves.

Outro ponto crítico é a latência. Embora os serviços em nuvem ofereçam grande poder de processamento, a distância física entre o dispositivo e o datacenter, somada ao tráfego na rede, pode introduzir atrasos que inviabilizam o uso em aplicações que demandam resposta imediata. Nesses casos, a adoção de arquiteturas híbridas, combinando processamento em borda e na nuvem, pode ser uma alternativa mais eficiente.

A segurança da informação também é um fator central. Dados em trânsito e armazenados em nuvem estão sujeitos a ameaças como interceptação, vazamento, alteração não autorizada e ataques direcionados a serviços de terceiros. Isso exige o uso de protocolos criptográficos robustos, autenticação forte, políticas de controle de acesso e conformidade com regulamentações específicas, como a Lei Geral de Proteção de Dados (LGPD) no Brasil e o Regulamento Geral sobre a Proteção de Dados (GDPR) na União Europeia.

O custo é outro aspecto a ser considerado. Embora o modelo de cobrança sob demanda seja atraente para projetos iniciais, a escalabilidade pode resultar em custos significativos à medida que o volume de dados e a complexidade do processamento aumentam. Além disso, operações que envolvem grandes transferências de dados ou consultas frequentes a bancos de dados na nuvem podem gerar despesas inesperadas, tornando essencial o monitoramento contínuo do consumo e a otimização dos recursos utilizados.

Outro desafio está relacionado à dependência de fornecedores (vendor lock-in). Muitos serviços de nuvem utilizam tecnologias proprietárias, o que pode dificultar ou encarecer a migração para outra plataforma no futuro. Essa limitação deve ser considerada no planejamento arquitetural, adotando-se soluções compatíveis com

padrões abertos e estratégias que evitem a dependência excessiva de um único provedor.

Por fim, questões de privacidade e conformidade legal precisam ser endereçadas desde o início do projeto. É fundamental garantir que os dados sejam armazenados e processados em conformidade com leis e regulamentações aplicáveis, o que pode incluir restrições quanto à localização física dos servidores e requisitos de auditoria.

A compreensão desses desafios é essencial para que a computação em nuvem seja utilizada de forma estratégica e segura em IoT. A próxima seção apresentará um panorama comparativo entre nuvem, névoa e borda, permitindo identificar os contextos ideais para cada abordagem e como integrá-las em soluções híbridas eficientes.

3.3 Estratégias para maximizar o uso da nuvem

O aproveitamento pleno do potencial da computação em nuvem em aplicações de Internet das Coisas requer mais do que apenas o envio de dados para servidores remotos. É necessário adotar estratégias que conciliem eficiência, segurança, custo e escalabilidade, garantindo que os benefícios superem as limitações naturais dessa abordagem.

Uma das práticas mais relevantes é o pré-processamento de dados na borda. Ao executar filtragem, compressão e agregação localmente, reduz-se significativamente o volume de informações transmitidas, economizando largura de banda e custos de armazenamento. Essa abordagem também contribui para minimizar a latência, uma vez que apenas eventos relevantes chegam à nuvem para processamento avançado e registro histórico.

Outra estratégia fundamental é a integração com serviços escaláveis e flexíveis, evitando o chamado vendor lock-in. A escolha de provedores que ofereçam APIs abertas e compatibilidade com padrões amplamente utilizados aumenta a liberdade para migrar ou integrar soluções conforme as necessidades do projeto evoluem, preservando a capacidade de inovação e adaptação tecnológica.

A otimização do armazenamento e análise de dados também é crucial. Isso inclui o uso de políticas de retenção inteligente, que descartam ou arquivam

informações de baixa relevância após um determinado período, e a adoção de mecanismos de indexação e consultas otimizadas, garantindo desempenho consistente mesmo com o crescimento do volume de dados.

No aspecto de segurança, recomenda-se a implementação de criptografia ponta a ponta e autenticação robusta, assegurando que a confidencialidade e integridade das informações sejam preservadas desde a coleta até o armazenamento e a análise. O uso de certificados digitais, protocolos seguros como HTTPS e MQTT sobre TLS (MQTTS), além de controle rigoroso de chaves e tokens de acesso, contribui para proteger os sistemas contra ameaças cibernéticas.

Por fim, a monitorização contínua dos custos e desempenho da infraestrutura em nuvem permite ajustes proativos, evitando desperdícios e garantindo que os recursos estejam dimensionados de acordo com as demandas reais da aplicação. Ferramentas de *monitoring* e *alerting* ajudam a identificar gargalos e otimizar a alocação de recursos.

Ao combinar essas práticas, é possível maximizar o retorno sobre o investimento em soluções baseadas em nuvem, garantindo alto desempenho, segurança e escalabilidade. Essas estratégias também facilitam a integração com outras camadas da arquitetura distribuída, criando ecossistemas IoT mais robustos e resilientes.

FIAP

REFERÊNCIAS

CRUZ, A.; SANTOS, R. Computação em Nuvem. **Universidade Federal do Rio de Janeiro – Grupo de Teleinformática e Automação (GTA)**, 2015. Disponível em: <https://www.gta.ufrj.br/ensino/eel879/trabalhos_vf_2015_2/OpenStack/introducao/nuvem.html>. Acesso em: 02 set. 2025.

FONSECA, Marcelo; OLIVEIRA, Luiz; SILVA, Roberta. Modelo de arquitetura distribuída em IoT com suporte à computação em borda, névoa e nuvem. **Revista Brasileira de Computação Aplicada**, v. 14, n. 3, p. 12–25, 2022.

GRAFANA LABS. Plataforma de visualização e monitoramento de dados. **Grafana Cloud**, 2025. Disponível em: <https://grafana.com/products/cloud/>. Acesso em: 02 set. 2025.

HIVEMQ. Console de gerenciamento de brokers MQTT em nuvem. **HiveMQ Cloud**, 2025. Disponível em: <https://www.hivemq.com/cloud/>. Acesso em: 02 set. 2025.

INFLUXDATA. Plataforma para armazenamento e análise de séries temporais. **InfluxDB Cloud**, 2025. Disponível em: <https://www.influxdata.com/products/influxdb-cloud/>. Acesso em: 02 set. 2025.

NODE-RED. Plataforma de desenvolvimento visual para lógica de aplicações IoT. **Node-RED**, 2025. Disponível em: <https://nodered.org/>. Acesso em: 02 set. 2025.

WOKWI. Simulador online de circuitos e microcontroladores. **Wokwi**, 2025. Disponível em: <https://wokwi.com/>. Acesso em: 02 set. 2025.